

designed chunks may yield insufficient or noisy context sent back by the retriever (due to either the design of the chunks or poor quality of the embeddings), resulting in the LLM generating incomplete or inaccurate answers, even when it has high-quality embeddings as a source.

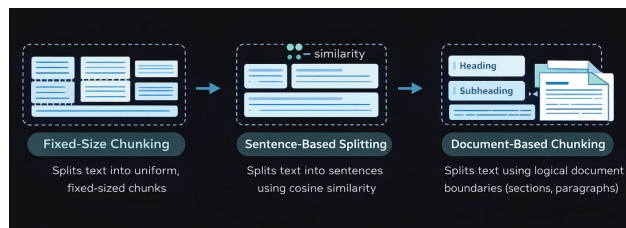


Figure 3: Chunking

Chunks that are too small lose any connected semantics and relationships, which may affect performance, whereas chunks that are too large introduce irrelevant context and waste tokens. A well-designed chunking strategy will allow each chunk to be retrievable, meaningful, and self-containing.

Fixed size chunking: The document is broken up into chunks that are characterised by the presence of tokens (single units of language), for example, in fixed size chunks. The advantage of this method is speed and ease of implementation, but the disadvantage is that it often breaks apart logical boundaries in a way that creates discontinuities in the represented data.

```
from typing import List
import re

# Split the text into units (words, in this case)
def word_splitter(source_text: str) -> List[str]:
    source_text = re.sub("\s+", " ", source_text) # Replace
    multiple whitespaces
    return re.split("\s", source_text) # Split by single
    whitespace

def get_chunks_fixed_size_with_overlap(text: str, chunk_size:
int, overlap_fraction: float = 0.2) -> List[str]:
    text_words = word_splitter(text)
    overlap_int = int(chunk_size * overlap_fraction)
    chunks = []
    for i in range(0, len(text_words), chunk_size):
        chunk_words = text_words[max(i - overlap_int, 0): i +
chunk_size]
        chunk = " ".join(chunk_words)
        chunks.append(chunk)
    return chunks
```

Sentence-based chunking: The sentences in the document are used as the points at which to divide it up into chunks. The

option of merging chunks into single ones based upon their similarity may be considered as well when implementing this strategy. This method provides better structural support than the previous method of chunking at fixed sizes; however, it can still be impeded by chunks created from conceptually multi-sentence pieces or lengthy explanations.

```
from typing import List
def recursive_chunking(text: str, max_chunk_size: int = 1000)
-> List[str]
    # Base case: if text is small enough, return as single
    chunk
    if len(text) <= max_chunk_size:
        return [text.strip()] if text.strip() else []
    # Try separators in priority order
    separators = ["\n\n", "\n", ".", " ", " "]

    for separator in separators:
        if separator in text:
            parts = text.split(separator)
            chunks = []
            current_chunk = ""

            for part in parts:
                # Check if adding this part would exceed the
                limit
                test_chunk = current_chunk + separator + part
                if current_chunk else part

                if len(test_chunk) <= max_chunk_size:
                    current_chunk = test_chunk
                else:
                    # Save current chunk and start new one
                    if current_chunk:
                        chunks.append(current_chunk.strip())
                    current_chunk = part

            # Add the final chunk
            if current_chunk:
                chunks.append(current_chunk.strip())

            # Recursively process any chunks that are still
            too large
            final_chunks = []
            for chunk in chunks:
                if len(chunk) > max_chunk_size:
                    final_chunks.extend(recursive_
chunking(chunk, max_chunk_size))
                else:
                    final_chunks.append(chunk)

    return [chunk for chunk in final_chunks if chunk]
```